Hoisting

- JavaScript hoisting allows accessing variables and functions before they are declared.
- This happens due to MEMORY ALLOCATION in execution context.
  - "*Before, code get executed memory allocation stat for all var & fun due to that we can access them before declaration.*"
- Hoisting applies to variable declarations made using `var`, `let`, `const`, and function declarations.
  - For `var`: The variable is hoisted but initialized as `undefined`.
  - For `let` and `const`: The variables are hoisted but are in a Temporal Dead Zone (TDZ), meaning they cannot be accessed before the actual declaration.
  - For `functions`: The entire function definition is hoisted, making the function usable before its declaration.
- Most of other programming language it throws an error it is not possible to access `variable` `fun` before declaration.

Example

1. Global Scope

- Here , We try to access `x` it will give `not defined error` because `Hoisting` happens only for those variable who declared by `var, let, const`.
- For `b` , it is declared inside function due to block scope not possible to access `b` outside the function.

```
console.log(x); // No hoisting, x is not defined
console.log(codeHoist); // codeHoist() {..}

x = 40; // global scope
function codeHoist() {
    a = 10;
    let b = 50;
}
codeHoist();
```

2. JavaScript var hoisting

- Here, `name` can be accessible its hoisted during memory allocation because it is declared as `var` keywords.
- the output is `undefined`

```
// var code (global)
console.log(name); // undefined
var name = 'India';
```

3. Function scoped variable

- Here, `name` is accessible due to functional scope. not possible to access outside the function.

```
// Function scoped
function fun() {
  console.log(name);  // Undefined
  var name = 'Mukul Latiyan';
}
fun();
```

4. JavaScript hoisting with Let & const

- Variables declared with `let , const` are hoisted but are placed in a Temporal Dead Zone (TDZ).
- The TDZ is the phase from the start of the block until the variable is declared and initialized. During this period, accessing the variable will throw a `ReferenceError`.
- The variable is only available after the declaration.

```
console.log(b);  // ReferenceError: Cannot access 'b' before initialization
let b = 20;
console.log(b);  // 20

console.log(c); // ReferenceError: Cannot access 'c' before initialization
const c = 30;
console.log(c);  // 30
```

Shortest Program in JS

- A JavaScript program can be an empty file, meaning no code at all.
- When you run JavaScript, the first thing that happens is the creation of the Global Execution Context.
- This context consists of:
  - Global Object: In the browser environment, this is the `window` object.
  - `this` keyword: In the global context, `this` refers to the global object (i.e., `window` in the browser).
- Global Scope :
  - Scope means Area or Boundaries
  - Is the outermost scope in JavaScript.
  - It includes anything that is not inside a function.
  - Code that is not within a function is said to be in the global space.

Undefined vs Not Defined

1. Undefined in JavaScript:

- When a variable or function has been declared, it is considered defined.
- It is kind of placeholder for a any type of value before that it is `undefined`.
- When the JavaScript engine starts executing the script, the first step is creating the Global Execution Context. During the Creation Phase, memory is allocated for variables and functions so its defined but their value is `undefined`.

```
console.log(x);  // Outputs: undefined
var x = 10;  // x is defined
console.log(x);  // Outputs: 10
```

2. Not defined in JavaScript:

- The variable or function has not been declared.
- When the JavaScript engine starts executing the script, value that we have access first it checked inside memory if its found than we can access that else get `ReferenceError`.

```
var a = 5;  // a is defined
console.log(a);  // Outputs: 5

console.log(b);  // ReferenceError: b is not defined
```